

Process and tools



KHOA CÔNG NGHỆ THÔNG TIN
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN

fit@hcmus

Parallelizing/optimizing a program

What part(s) should be parallelized/optimized?

- Measure times of parts to decide
- When optimizing (after parallelizing), can measure times of GPU activities quickly:
`nvprof --print-gpu-trace ./a.out`

1. Analyze

2. Design

3. Implement

4. Evaluate

How to parallelize/optimize?

- Each loop will create a new version based on previous versions
- We should go step by step, from sequential to parallel, from parallel to optimized parallel

Does the idea work?
If not, do you know why?

Parallelizing/optimizing a program

What part(s) should be parallelized/optimized?

- Measure times of parts to decide
- When optimizing (after parallelizing), can measure times of GPU activities quickly:
`nvprof --print-gpu-trace ./a.out`

1. Analyze

2. Design

3. Implement

4. Evaluate

How to go through this process as well as possible?

Some advices:

- **Keep the mind still**
- Keep the code clean
- Code fast or slow?
- Use a good editor and learn how to use it efficiently

How to parallelize/optimize?

Does the idea work?
If not, do you know why?

General optimization guidelines

- Expose enough independent tasks to utilize GPU hardware resources
 - ▣ Expose enough blocks to utilize SMs
 - ▣ In each SM, expose enough independent instructions (coming from the same warp, or from different warps) to utilize execution pipelines, hide latency
- Access DRAM efficiently
 - ▣ Don't let threads in the same warp access scattered addresses in DRAM
 - ▣ Use SMEM to reduce DRAM accesses, as well as to access DRAM efficiently
- Reduce warp divergence

Sequential version

- If we haven't had the sequential version (as in the project), we will create it by going through similar steps but we will not have “analyze” step
 - ▣ **Design:** What steps to compute output from input?
 - ▣ **Implement** (based on the design)
 - ▣ **Evaluate:** Is the implementation correct?
How much time does it take to run?

TOOLS

Colab Notebook / Kaggle Notebook

- Run on cloud machines with **free GPUs**
 - ▣ Kaggle often provides better GPU than Colab
- Allow us to write and run Python code, write and run OS (Linux) commands, write Markdown text; all in one single file (.ipynb)
 - ▣ Colab interface is more convenient than Kaggle (e.g. Colab has “table of contents”) •
- We will use these tools to write down our process of speeding up a program by parallelizing/optimizing

Colab Notebook / Kaggle Notebook

- ❑ To use these tools efficiently, we need to learn keyboard shortcuts
- ❑ Where to find keyboard shortcuts?
 - ❑ Colab Notebook: “Tools” → “Keyboard shortcuts”
 - ❑ Kaggle Notebook: No document about keyboard shortcuts!
 - ❑ The secret is: its keyboard shortcuts are similar to Jupyter Notebook, because Kaggle Notebook is modified from Jupyter Notebook (Colab Notebook is also modified from Jupyter Notebook, but it is modified quite a lot and its keyboard shortcuts are quite different from Jupyter Notebook)

Python

- ☐ All of you have known about Python But how much?
- ☐ Let's do a quiz to find out ...
- ☐ In this course, we want to speed up a Python program → we will need to know a little bit about how Python works under the hood to understand why sometimes Python code runs so slowly

Python

- ☐ Why did Guido van Rossum create Python programming language, although C had already existed at that time?
- ☐ Guido van Rossum: “I realized that the development of system administration utilities in C was taking too long”

Why Python write faster

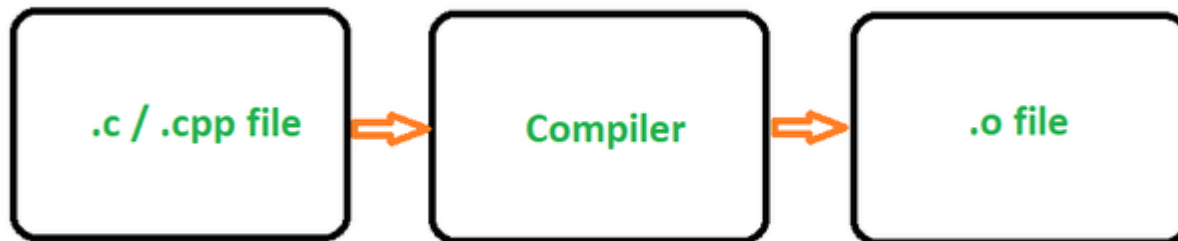
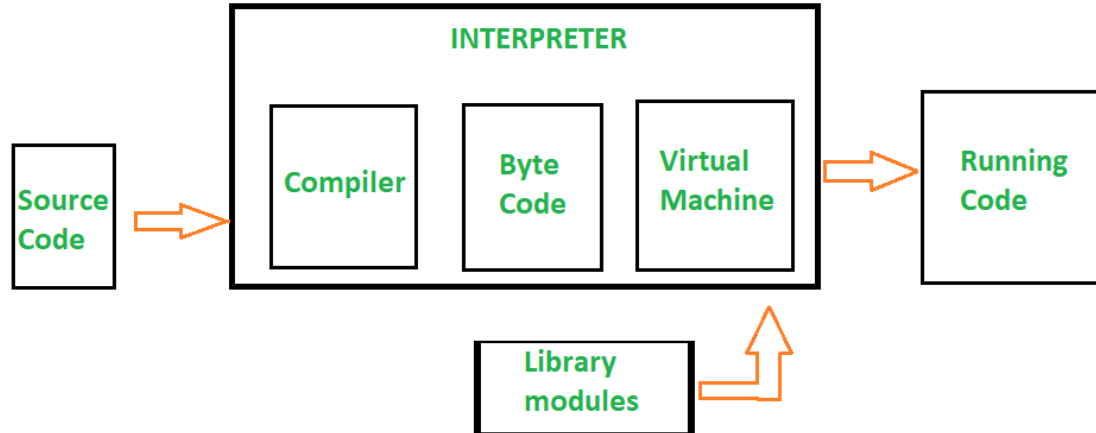
- ❑ **Interpreted language:** no need to compile all code before running; instead, programmers can code some lines of code and run immediately, and can continue to code and run from previous running state
- ❑ **Dynamic typing**

```
a = 1
...
a = 'hello'

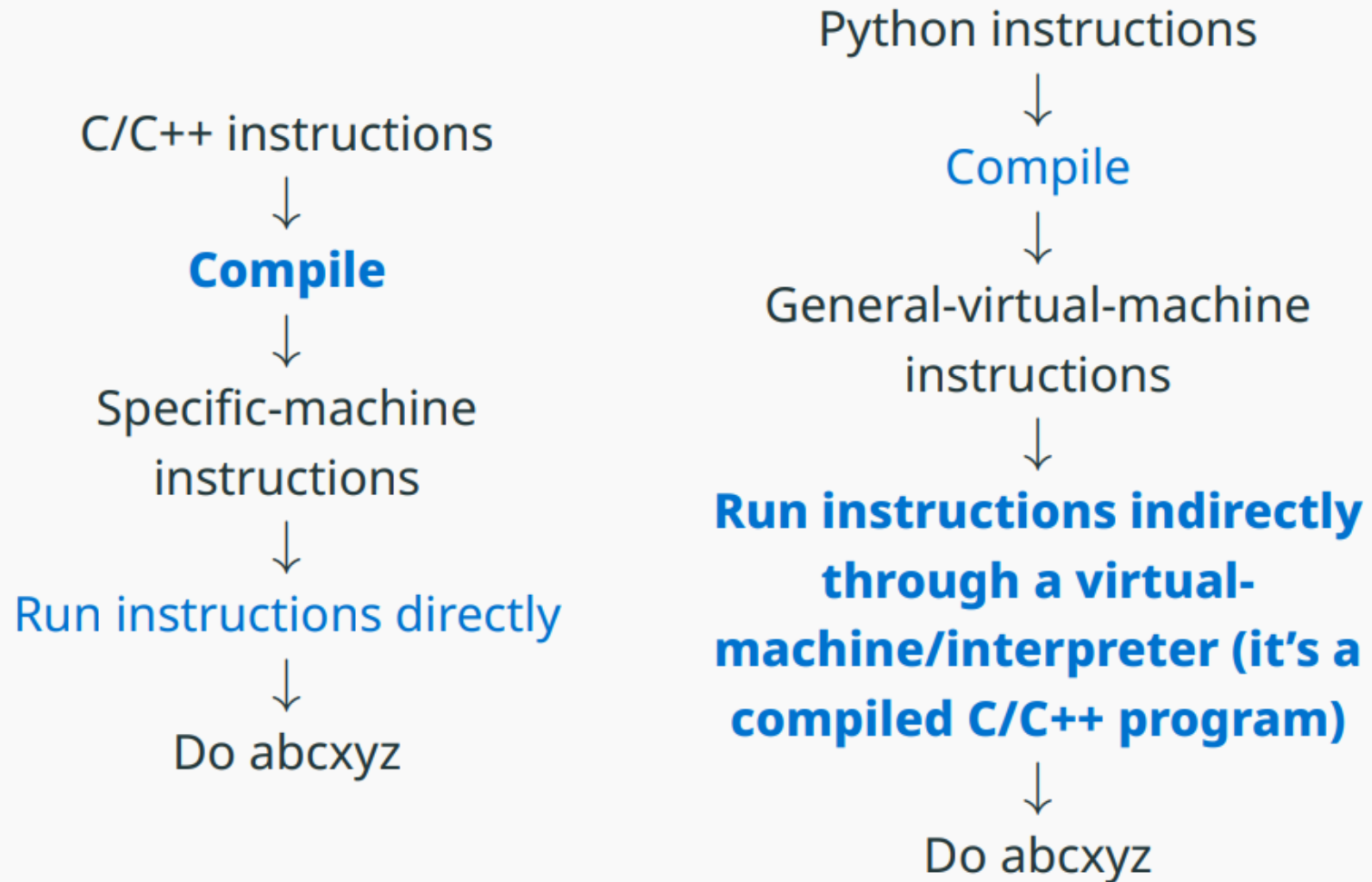
def add_two_things(a, b):
    return a + b

# Can call add_two_things
# with 2 arbitrary objects
# having + operation
```
- ❑ **Automatic memory management:** programmers don't have to spend time doing malloc and free memories manually as in C/C++

Why Python write faster



Why Python write faster



Why Python run slower

- **Interpreted language:** interpreted languages **often run slower** than compiled languages, because in compiled languages, before running, compiler will look at all source code and do optimizations
- **Dynamic typing:** to achieve this, a variable in Python is just a pointer pointing to an object, and this object contains not only value but also meta info such as data type, ...; when do an operation with 2 objects, Python interpreter first have to **spend time** opening these objects to identify data type
- **Automatic memory management:** it also costs **extra work and time** to know when to free allocated memories

Numpy

- Is a Python library which helps speed up pure Python code when working with numerical data?
- Provides Numpy array – a data structure to compactly store a list of values (its appearance to users can be 1D, 2D, ... array) with the same data type and with the fixed number of values
 - ▣ A Numpy array stores values as an array in C/C++: it just stores bare values and these values lie next to each other in memory
 - ▣ Python list: each value is contained in an object, one object can lie anywhere in memory
- Also provides operations which work directly with Numpy arrays; behind the scene, Numpy use precompiled optimized C/C++ codes to do loops

Numpy

- ☐ Numpy is a Python lib which helps speed up Python program when working with large amount of numerical data
- ☐ Numpy helps speed up by providing:
 - ☐ Numpy array – a Python object wrapping around a C/C++ array
 - ☐ Numpy operations (+, -, *, /, ...) on Numpy arrays; these operations are precompiled optimized C/C++ codes
- ☐ However:
 - ☐ Numpy only run sequentially on CPU
 - ☐ There are cases of preprocessing data which are difficult to express using Numpy operations; using loops is easier but ...

Numba

- Numba is a also Python lib which helps speed up Python program when working with large amount of numerical data
- But Numba solves Numpy's disadvantages
 - ▣ Numba allows to use loops with Numpy arrays
 - ▣ Numba allows to run parallel on CPU as well as GPU